Computational Gastronomy — Assignment 2

Question 2: Size-Controlled Random Control (Null Models)

Monsoon 2026

Problem Statement

Create size-controlled random control of the above 10,000 recipes assuming them to be a cuisine.

- **(a)** Create a size-controlled cuisine by making a replica of each recipe. Plot the recipe size distribution of the original cuisine and the randomly created recipes in 10 sets. `[5]`
- **(b)** Create a size-controlled cuisine by using 'inverse transformation'. Plot the recipe size distribution of the original cuisine and the randomly created recipes (10 times the original). `[5]`

In [1]:
```
import sys
import os
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Styling
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style
plt.rcParams['font.sans-serif'] = ['DejaVu Sans', 'Arial', 'Helvetica']
plt.rcParams['axes.edgecolor'] = '#cccccc'
plt.rcParams['axes.linewidth'] = 0.8

DATA_PATH = Path('../Group_23_Assignment1/Q1b_ingredients_ner.csv')
RANDOM_SEED = 20260929
np.random.seed(RANDOM_SEED)
```

In [2]:
```
# Load NER ingredients
ner_df = pd.read_csv(DATA_PATH, keep_default_na=False)
valid_ner = ner_df[ner_df['Ingredient Name'].str.strip() != '']

# Recipes as sets of unique ingredients
recipes_dict = valid_ner.groupby('Recipe ID')['Ingredient Name'].apply(lambda s
recipe_ids = sorted(list(recipes_dict.keys()))
N = len(recipe_ids)

# Vocabulary of unique ingredients
vocabulary = sorted(list(set(valid_ner['Ingredient Name'])))
V = len(vocabulary)

# Original recipe sizes
```

Loading [MathJax]/extensions/Safe.js

```
original_sizes = np.array([len(recipes_dict[rid]) for rid in recipe_ids])

print(f"Total recipes: {N:,}")
print(f"Vocabulary size |V|: {V:,}")
print(f"Recipe size statistics (Original Cuisine):")
print(f"  • Mean:   {original_sizes.mean():.4f}")
print(f"  • Median: {np.median(original_sizes):.1f}")
print(f"  • Std:    {original_sizes.std():.4f}")
print(f"  • Min:    {original_sizes.min()}, Max: {original_sizes.max()}")
```

```
Total recipes: 10,000
Vocabulary size |V|: 2,425
Recipe size statistics (Original Cuisine):
  • Mean:   9.7190
  • Median: 9.0
  • Std:    4.0825
  • Min:    1, Max: 31
```

Q2(a) — Size-Controlled Cuisine via Recipe Replicas

Methodology:

1. In this model, for every recipe r_i in the original cuisine with size $s_i = |r_i|$, we construct a random synthetic replica r_i' of identical size s_i.
2. The s_i ingredients in r_i' are sampled uniformly at random from the total ingredient vocabulary V without replacement.
3. Because every single recipe's size is preserved exactly ($|r_i'| = |r_i|$), the recipe size distribution of the random cuisine matches the original empirical distribution identically.
4. We generate **10 independent random cuisines** (sets 1 to 10) using different random seeds and compare their size distributions to the original cuisine.

In [3]:

```
num_replica_sets = 10
replica_sets_sizes = []

for k in range(num_replica_sets):
    rng = np.random.default_rng(seed=RANDOM_SEED + k * 100)
    # For each recipe of size s, sample s ingredients uniformly from vocabulary
    replica_sizes = [len(rng.choice(V, size=s, replace=False)) for s in origina
    replica_sets_sizes.append(np.array(replica_sizes))

# Plot recipe size distribution: Original vs 10 replica sets
fig, ax = plt.subplots(figsize=(10, 6), dpi=300)

# Compute size range
max_size = max(original_sizes.max(), max(s.max() for s in replica_sets_sizes))
bins = np.arange(0.5, max_size + 1.5, 1)
bin_centers = np.arange(1, max_size + 1)

# Plot 10 random replica sets
palette = plt.cm.tab10(np.linspace(0, 1, 10))
for k in range(num_replica_sets):
    counts, _ = np.histogram(replica_sets_sizes[k], bins=bins)
    ax.plot(bin_centers, counts, color=palette[k], alpha=0.6, linewidth=1.5,
            label=f'Random Replica Set {k+1}' if k < 3 or k == 9 else "")
```

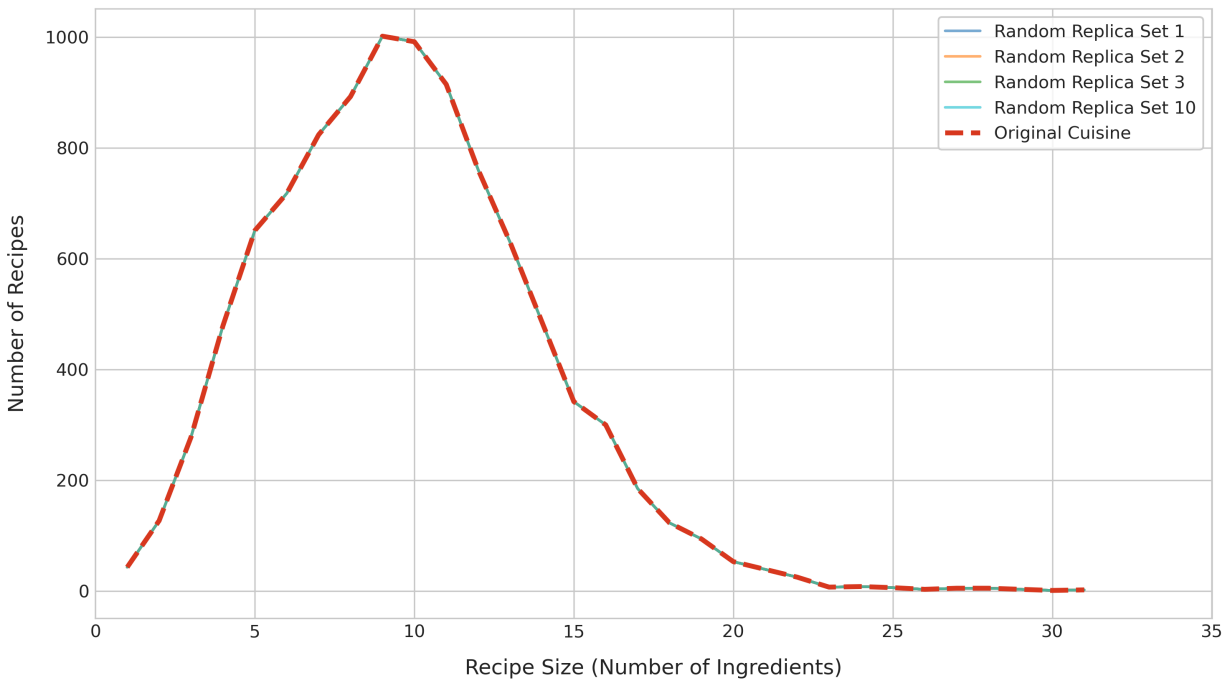
Loading [MathJax]/extensions/Safe.js

```
# Plot original cuisine with prominent styling
orig_counts, _ = np.histogram(original_sizes, bins=bins)
ax.plot(bin_centers, orig_counts, color='#d9381e', linewidth=2.8, linestyle='--
        label='Original Cuisine', zorder=10)

ax.set_title('Q2(a): Recipe Size Distribution — Original Cuisine vs 10 Replica
ax.set_xlabel('Recipe Size (Number of Ingredients)', fontsize=12, labelpad=10)
ax.set_ylabel('Number of Recipes', fontsize=12, labelpad=10)
ax.set_xlim(0, 35)
ax.legend(frameon=True, facecolor='white', framealpha=0.9, fontsize=10)
plt.tight_layout()

plt.savefig('Q2a_recipe_size_distribution_replica_10sets.png', dpi=300)
plt.show()

# Verification check
print("Size distribution difference check:")
for k in range(num_replica_sets):
    diff = np.max(np.abs(original_sizes - replica_sets_sizes[k]))
    print(f"  • Replica Set {k+1}: Max deviation in recipe sizes = {diff}")
```




```
Size distribution difference check:
  • Replica Set 1: Max deviation in recipe sizes = 0
  • Replica Set 2: Max deviation in recipe sizes = 0
  • Replica Set 3: Max deviation in recipe sizes = 0
  • Replica Set 4: Max deviation in recipe sizes = 0
  • Replica Set 5: Max deviation in recipe sizes = 0
  • Replica Set 6: Max deviation in recipe sizes = 0
  • Replica Set 7: Max deviation in recipe sizes = 0
  • Replica Set 8: Max deviation in recipe sizes = 0
  • Replica Set 9: Max deviation in recipe sizes = 0
  • Replica Set 10: Max deviation in recipe sizes = 0
```

Loading [MathJax]/extensions/Safe.js

Q2(b) — Size-Controlled Cuisine via Inverse Transformation Sampling

Methodology:

1. **Inverse Transform Sampling** allows generating random variates from an arbitrary discrete empirical cumulative distribution function (CDF).
2. Given empirical size probabilities $P(S = s_k) = p_k$ and CDF $F(s) = \sum_{s_k \le s} p_k$:
   - Generate standard uniform random variate $U \sim \text{Uniform}(0, 1)$.
   - Find the quantile $s = F^{-1}(U) = \min \{s_k \mid F(s_k) \ge U\}$.
3. We generate **$100,000$ recipes** ($10 \times$ the original size of $10,000$) using inverse transformation sampling.
4. Each sampled size s_j is then populated by drawing s_j ingredients uniformly at random from the vocabulary without replacement.
5. We plot and compare the normalized recipe size probability mass distributions (PDF) and cumulative distribution functions (CDF) of the original cuisine and the $10\times$ inverse-transformed random cuisine.

In [4]:
```
# 1. Compute Empirical CDF of recipe sizes
unique_sizes, counts = np.unique(original_sizes, return_counts=True)
empirical_pdf = counts / len(original_sizes)
empirical_cdf = np.cumsum(empirical_pdf)

# 2. Inverse Transformation Sampling for 10x recipes (100,000 recipes)
N_10x = 10 * N
rng_inv = np.random.default_rng(seed=RANDOM_SEED + 777)
u_uniform = rng_inv.uniform(0.0, 1.0, size=N_10x)

# Searchsorted finds the index where u <= cdf
sampled_size_indices = np.searchsorted(empirical_cdf, u_uniform)
sampled_sizes_10x = unique_sizes[sampled_size_indices]

# Compare summary statistics
print(f"Generated {N_10x:,} recipes via Inverse Transform Sampling:")
print(f"  Original Cuisine (N = {N:,}):")
print(f"    Mean: {original_sizes.mean():.4f}, Std: {original_sizes.std():.4f}
print(f"  Random Cuisine 10x (N = {N_10x:,}):")
print(f"    Mean: {sampled_sizes_10x.mean():.4f}, Std: {sampled_sizes_10x.std(

# 3. Two-panel visualization: (Left) Size Distribution PDF, (Right) Size Cumula
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6), dpi=300)

max_s = max(original_sizes.max(), sampled_sizes_10x.max())
bins_inv = np.arange(0.5, max_s + 1.5, 1)
centers_inv = np.arange(1, max_s + 1)

# Left: Probability Density Function (Normalized Histogram)
orig_p, _ = np.histogram(original_sizes, bins=bins_inv, density=True)
inv_p, _ = np.histogram(sampled_sizes_10x, bins=bins_inv, density=True)

ax1.plot(centers_inv, orig_p, color='#1f77b4', linewidth=2.5, marker='o', marke
_inv, inv_p, color='#ff7f0e', linewidth=2.2, linestyle='--', ma
ax1.set_title('Probability Distribution P(Size = s)', fontsize=13, fontweight=
```

Loading [MathJax]/extensions/Safe.js

```
ax1.set_xlabel('Recipe Size (Number of Ingredients)', fontsize=11)
ax1.set_ylabel('Probability Density', fontsize=11)
ax1.set_xlim(0, 35)
ax1.legend(frameon=True, facecolor='white', fontsize=10)

# Right: Cumulative Distribution Function (CDF)
orig_sorted = np.sort(original_sizes)
orig_cdf_vals = np.arange(1, len(orig_sorted) + 1) / len(orig_sorted)

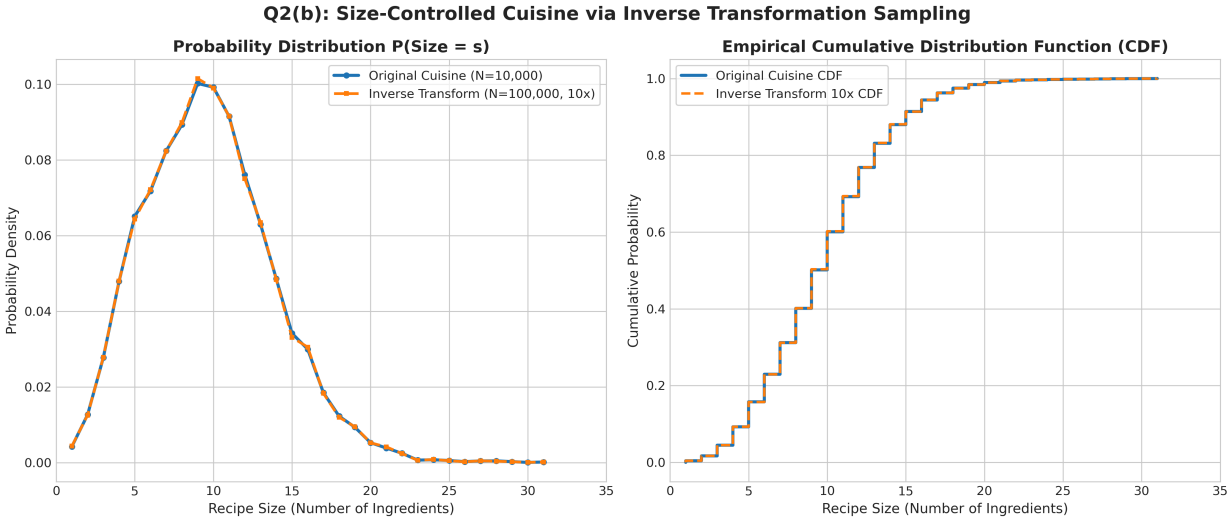
inv_sorted = np.sort(sampled_sizes_10x)
inv_cdf_vals = np.arange(1, len(inv_sorted) + 1) / len(inv_sorted)

ax2.step(orig_sorted, orig_cdf_vals, where='post', color='#1f77b4', linewidth=
ax2.step(inv_sorted, inv_cdf_vals, where='post', color='#ff7f0e', linewidth=2.(
ax2.set_title('Empirical Cumulative Distribution Function (CDF)', fontsize=13,
ax2.set_xlabel('Recipe Size (Number of Ingredients)', fontsize=11)
ax2.set_ylabel('Cumulative Probability', fontsize=11)
ax2.set_xlim(0, 35)
ax2.legend(frameon=True, facecolor='white', fontsize=10)

plt.suptitle('Q2(b): Size-Controlled Cuisine via Inverse Transformation Samplir
plt.tight_layout()

plt.savefig('Q2b_recipe_size_distribution_inverse_transform_10x.png', dpi=300)
plt.show()
```

```
Generated 100,000 recipes via Inverse Transform Sampling:
  Original Cuisine (N = 10,000):
    Mean: 9.7190, Std: 4.0825, Median: 9.0
  Random Cuisine 10x (N = 100,000):
    Mean: 9.7117, Std: 4.0773, Median: 9.0
```



Question 2 Deliverables Summary

Deliverable File	Description	Question
`Q2a_recipe_size_distribution_replica_10sets.png`	Size distribution plot comparing original cuisine against 10 replica	Q2(a)

Loading [MathJax]/extensions/Safe.js

Deliverable File	Description	Question
	random control sets	
`Q2b_recipe_size_distribution_inverse_transform_10x.png`	Normalized PDF and step CDF plots comparing original cuisine against 10x (100,000 recipes) inverse transformation control	Q2(b)

Loading [MathJax]/extensions/Safe.js